

```

const express = require('express');
const axios = require('axios');
const crypto = require('crypto');
const bodyParser = require('body-parser');

const app = express();

// PayPal sends JSON, we need raw body for verification
app.use(bodyParser.json({
  verify: (req, res, buf) => { req.rawBody = buf; }
}));

const {
  PAYPAL_CLIENT_ID,
  PAYPAL_SECRET,
  PAYPAL_WEBHOOK_ID,
  TREXITY_API_KEY,
  PORT = 3000
} = process.env;

const PAYPAL_API = 'https://api.paypal.com'; // live

async function getPayPalToken() {
  const auth =
    Buffer.from(`${PAYPAL_CLIENT_ID}:${PAYPAL_SECRET}`).toString('base64');
  const { data } = await axios.post(`${PAYPAL_API}/v1/oauth2/token`,
    'grant_type=client_credentials',
    { headers: { Authorization: `Basic ${auth}`, 'Content-Type': 'application/x-www-
form-urlencoded' } }
  );
  return data.access_token;
}

async function verifyWebhook(req) {
  try {
    const token = await getPayPalToken();
    const { data } = await axios.post(`${PAYPAL_API}/v1/notifications/verify-
webhook-signature`, {
      transmission_id: req.headers['paypal-transmission-id'],
      transmission_time: req.headers['paypal-transmission-time'],
      cert_url: req.headers['paypal-cert-url'],
      auth_algo: req.headers['paypal-auth-algo'],
      transmission_sig: req.headers['paypal-transmission-sig'],
      webhook_id: PAYPAL_WEBHOOK_ID,
      webhook_event: req.body
    }, { headers: { Authorization: `Bearer ${token}`, 'Content-Type':
'application/json' } });
    return data.verification_status === 'SUCCESS';
  } catch (e) {
    console.error('Verify error', e.response?.data || e.message);
    return false;
  }
}

app.post('/webhook/paypal', async (req, res) => {
  console.log('Webhook received:', req.body.event_type);

  const verified = await verifyWebhook(req);
  if (!verified) {

```

```

    console.log('Webhook verification failed');
    return res.status(400).send('Invalid');
  }

  if (req.body.event_type === 'PAYMENT.CAPTURE.COMPLETED') {
    const capture = req.body.resource;
    const custom = capture.custom_id || '';

    console.log('Payment:', capture.id, 'Amount:', capture.amount?.value, 'Custom:',
    custom);

    // Parse custom_id: pickup|dropoff|phone
    const [pickup, dropoff, phone] = custom.split('|');

    if (pickup && dropoff) {
      try {
        const trexityPayload = {
          pickup_address: pickup,
          dropoff_address: dropoff,
          // Edmonton default - adjust as needed
          pickup_city: "Edmonton",
          pickup_province: "AB",
          dropoff_city: "Edmonton",
          dropoff_province: "AB",
          contact_phone: phone || "",
          reference: `PayPal ${capture.id}`,
          // Add package details - customize for your business
          package_type: "parcel"
        };

        const { data } = await axios.post('https://api.trexity.com/v1/shipments',
        trexityPayload, {
          headers: { Authorization: `Bearer ${TREXITY_API_KEY}`, 'Content-Type':
          'application/json' }
        });

        console.log('Trexity booked:', data.id || data.tracking_number);
      } catch (err) {
        console.error('Trexity error:', err.response?.data || err.message);
      }
    } else {
      console.log('No pickup/dropoff in custom_id, skipping Trexity');
    }
  }

  res.status(200).send('OK');
});

app.get('/', (req, res) => res.send('Aether PayPal → Trexity webhook is running'));

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```